

Git Basics

Important Git commands explained in very simple words — with easy examples

Think of Git like this:

Git is like a **save-game system** for your project. It keeps checkpoints of your work so you can understand what changed.

Git and GitHub

Git is a tool on your computer that tracks changes in your project.

GitHub is an online place where you can store and share Git repositories.

Kid example:

Git is your notebook's **save system**. GitHub is like an **online cupboard** where you keep the notebook safely.

1. First-Time Setup

Tell Git your name and email. Usually this is done once on a computer.

```
git config --global user.name "Your Name"
```

```
git config --global user.email "you@example.com"
```

Git now knows who is making the commits.

2. Start Git — git init

Use this inside your project folder to make it a Git repository.

```
git init
```

Example: Open the terminal inside your **MyWebsite** folder and run **git init**. Git starts tracking the project.

Checking and Saving Your Work

The basic idea: **Check** → **Add** → **Commit**

3. Check Changes — git status

Ask Git: “**What has changed?**”

```
git status
```

It can show new files, modified files, and files prepared for a commit.

4. Prepare Changes — git add

Choose the changes you want to put into the next checkpoint.

```
git add index.html
```

To prepare everything:

```
git add .
```

Kid example:

You have 5 drawings on a table. **git add .** means putting all 5 drawings into the box you are preparing to save.

5. Save a Checkpoint — git commit

A commit saves the changes you selected and gives the checkpoint a message.

```
git commit -m "Added homepage"
```

Kid example:

You finish a drawing and write “**Finished the house**” on the back. The message tells you what this saved version contains.

6. See Your Saved History — git log

See the commits you made before.

```
git log
```

Short version:

```
git log --oneline
```

7. See Exact Changes — git diff

See what is different from the last saved version.

```
git diff
```

Working with GitHub

Git on your computer + GitHub online

8. Connect Your Project to GitHub

After creating a repository on GitHub, connect your local project to it.

```
git remote add origin https://github.com/USERNAME/REPOSITORY.git
```

Check the connection:

```
git remote -v
```

Simple idea:

Your computer has the project. GitHub is the online cupboard. The remote command tells Git where the cupboard is.

9. Send Work to GitHub — git push

Push means **send your local commits to GitHub**.

First push:

```
git push -u origin main
```

Later pushes:

```
git push
```

10. Get Work from GitHub — git pull

Pull means **bring the latest changes from GitHub to your computer**.

```
git pull
```

Example: Your friend changed the project on GitHub. You run **git pull** to get those changes.

11. Copy a GitHub Project — git clone

Clone downloads a copy of an existing repository to your computer.

```
git clone https://github.com/USERNAME/REPOSITORY.git
```

Kid example:

Your friend gives you a copy of their notebook. Now you have your own copy to work on.

Branches and Team Work

A branch gives you a safe side path for new work

12. Create a Branch

A branch lets you try a feature without immediately changing the main branch.

```
git branch feature-login
```

Move to the branch:

```
git switch feature-login
```

Create and move in one command:

```
git switch -c feature-login
```

13. Merge a Branch

When your feature is ready, combine it with the main branch.

```
git switch main  
git merge feature-login
```

Kid example:

You make a new page in a separate notebook. When it is ready, you combine that work with the main notebook.

14. Rename a File — git mv

Rename a file while telling Git about the change.

```
git mv old.html new.html
```

15. Remove a File — git rm

Remove a file and tell Git to track that removal.

```
git rm old.html
```

16. Restore an Uncommitted Change

Restore a file to the last committed version.

```
git restore filename
```

Be careful: this can remove your uncommitted changes in that file.

Git Cheat Sheet

Keep this page beside you while practising

Command	Easy meaning	Example use
git init	Start Git in a project	Start tracking MyWebsite
git status	Check what changed	See changed files
git add .	Prepare all changes	Prepare files for commit
git commit -m "..."	Save a checkpoint	Save: Added login page
git log	See saved history	View old commits
git diff	See exact changes	Compare current work
git clone ...	Copy a repository	Get a project from GitHub
git pull	Get latest GitHub work	Update your local project
git push	Send commits to GitHub	Upload your saved work
git branch	Create/list branches	Start a feature branch
git switch ...	Move between branches	Switch to feature-login
git merge ...	Combine branches	Merge feature into main
git restore ...	Undo an uncommitted file change	Restore index.html
git mv ...	Rename a file	Rename old.html
git rm ...	Remove a file	Delete old.html

The most important workflow:

1. **git status** → Check changes
2. **git add .** → Prepare changes
3. **git commit -m "message"** → Save a checkpoint
4. **git push** → Send it to GitHub

Memory Trick

CHECK → ADD → COMMIT → PUSH

Look at your work → pack the changes → save a checkpoint → send it online.